

Embedded Systems Architectures - Laboratory Guide 3

Basic Input/Output

Academic Year 2025/2026

Authors: Arnaldo Oliveira, Fábio Coutinho

1. Objectives

- Configuration and use of digital output ports
- Configuration and use of digital input ports (polling and interrupt)
- Configuration and use of queues for communication between tasks
- Configuration and generation of PWM signals
- Configuration and reading of analog input ports

2. Introduction

In the Internet of Things (IoT) and embedded systems, the ability to control and read electrical signals via digital and analog input/output ports is fundamental to creating intelligent, interactive devices. Knowing how to program these ports allows a microcontroller to communicate with sensors, actuators, and other peripherals, transforming signals from the physical world into digital data that can be processed and transmitted, or converting digital commands into concrete actions, such as turning on a light, moving a motor, or regulating the intensity of an LED. This ability not only provides direct control over the environment but also enables automation, real-time monitoring, and the integration of multiple devices into an IoT network. Understanding the operation and programming of I/O ports is therefore essential for developing robust, efficient, and responsive solutions, allowing creators to transform ideas into functional applications that intelligently interact with the world around them.

In this class, we will explore using the kit based on the ESP32-C6 module to perform operations such as controlling an LED, reading the state of a push button, and monitoring the voltage of a resistive divider implemented with a potentiometer. These are basic I/O operations, but they are fundamental for building an integrated IoT system.

2.1 Required Components

In addition to the components already provided, the following ones are required for this work and will be provided at the beginning of the class:

- 10 kOhm potentiometer with knob
- 180 Ohm resistor
- LED (Light Emitting Diode)
- Push button

The three circuits specified in Figure 1 should be implemented on the breadboard.

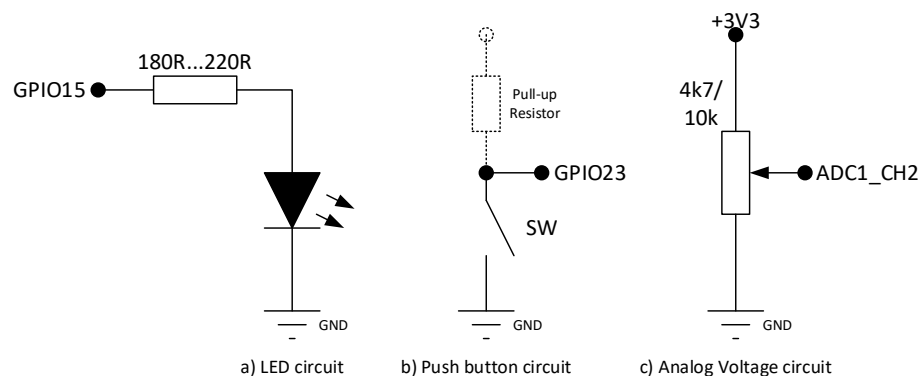


Figure 1 – Interface circuits with the ESP32-C6 kit to be used in this work.

Warning! Be careful and check the connections on the breadboard before powering the ESP32. Be particularly cautious when checking the connections with the ESP32-C6 kit.

3. Practical Work

The practical work in this class is based on the Reference API for the ESP32-C6 peripherals and the ESP32-C6 Technical Reference Manual:

https://documentation.espressif.com/esp32-c6_technical_reference_manual_en.pdf

3.1 Controlling an LED

The most basic step in any embedded system is to program a digital output to alternate between the "1" and "0" states with a given period. Next, the "skeleton" of a functional program is provided, which, combined with the circuit in Figure 1a), allows the respective LED to be activated periodically.

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"

#define BLINK_GPIO    GPIO_NUM_15    // GPIO15 on ESP32-C6

void app_main(void)
{
    gpio_config_t ioCfg = { // configure GPIO15 as output
        .pin_bit_mask = 1ULL << BLINK_GPIO, // bit mask for GPIO15
        .mode = GPIO_MODE_OUTPUT,
        .pull_up_en = GPIO_PULLUP_DISABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type = GPIO_INTR_DISABLE
    };
    gpio_config(&ioCfg);

    bool level = false;
    while (1)
    {
        gpio_set_level(BLINK_GPIO, level);
        level = !level; // invert for next toggle
        vTaskDelay(pdMS_TO_TICKS(1000)); // delay 1 second
    }
}
```

Troubleshooting: If you encounter problems getting the program to compile due to unmet dependencies, you can edit the *CMakeLists.txt* file located in your root folder (the one containing your "main" folder) using the following code block. Since there are no **REQUIRES** or **PRIV_REQUIRES**, you do not declare explicit dependencies on any other component, therefore all components defined in the project will be compiled and linked normally. In development, it is recommended that dependencies be explicitly identified using **REQUIRES** and **PRIV_REQUIRES** in the **idf_component_register**.

```
idf_component_register(SRCS "main.c"
                       INCLUDE_DIRS ".")
```

Consulting the GPIO & RTC section of the ESP32-C6 API Reference available at the web page <https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/api-reference/peripherals/gpio.html>), analyze the program and test it on the board, observing the LED "blinking" with a period of 2 seconds.

Question 1: Change the code to an update period of 20 milliseconds and observe the signal produced by the board on the oscilloscope, confirming both the intended frequency and the duty cycle of the signal.

Question 2: Restore the original period (2 seconds) and add a periodic printout of the LED state so that the following information is printed to the terminal.

```
~$ GPIO 15 set to 1
~$ GPIO 15 set to 0
~$ GPIO 15 set to 1
~$ GPIO 15 set to 0
~$ ...
```

3.2 Reading the state of a push button (polling)

Starting from the circuit in Figure 1b), we now want to read the state of a push button.

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
```

```

#include "freertos/task.h"
#include "driver/gpio.h"

#define INPUT_GPIO  GPIO_NUM_23  // choose an available board GPIO

void app_main(void)
{
    // Configure the GPIO as input with pull-up enabled
    gpio_config_t ioCfg = {
        .pin_bit_mask = 1ULL << INPUT_GPIO, // selected GPIO bit mask
        .mode = GPIO_MODE_INPUT,
        .pull_up_en = GPIO_PULLUP_ENABLE, // enable internal pull-up
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type = GPIO_INTR_DISABLE
    };
    gpio_config(&ioCfg);

    while (1)
    {
        int level = gpio_get_level(INPUT_GPIO); // read the pin state
        // Include here the code to print the button state

        vTaskDelay(pdMS_TO_TICKS(1000)); // check every 1s
    }
}

```

Question 3: Analyze the configuration options and complete the code so that the state of the push button is displayed on the monitor according to the following example:

```

~$ GPIO 23 read 1
~$ GPIO 23 read 1
~$ GPIO 23 read 1
~$ GPIO 23 read 0
~$ ...

```

Question 4: Proceed with compiling and running the developed code and verify its operation. What can you say about the operating logic of the push button? Is it direct or inverted?

Question 5: What is the purpose of the resistor connected to the push button and identified in the circuit of Figure 1b) in dashed lines? Why, although you did not assemble this resistor, is it internally connected? Why is it known as a pull-up resistor?

3.3 Reading the state of a push button (interrupt)

When using interrupts in embedded systems, the challenge of securely transferring information between the interrupt service routine (ISR) and the main application always arises. Implementing operations such as printing within an ISR is not recommended, as it can delay other important events. Queues offer an efficient and secure mechanism for this communication. The ISR can quickly queue a small piece of data, such as an event signal or a sensor reading, and return it immediately. Later, a FreeRTOS task, running outside the interrupt context, retrieves this information from the queue and processes it without time constraints. In this way, ISRs remain lightweight while ensuring reliable and synchronized data transfer between interrupt events and application logic.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "driver/gpio.h"

#define INPUT_GPIO    GPIO_NUM_23    // input pin example

static QueueHandle_t gpioEvtQueue = NULL;

// ISR handler
static void IRAM_ATTR GPIOISRHandler(void* arg)
{
    uint32_t gpioNum = (uint32_t)arg;
    // Send GPIO number to the queue (from ISR context)
    xQueueSendFromISR(gpioEvtQueue, &gpioNum, NULL);
}
```

```

// Task that handles GPIO events
static void GPIONTask(void* arg)
{
    uint32_t ioNum;
    for( ; ; )
    {
        if (xQueueReceive(gpioEvtQueue, &ioNum, portMAX_DELAY))
        {
            // Include here the code to print the GPIO number
            // that is the source of the interrupt
        }
    }
}

void app_main(void)
{
    // Configure the GPIO as input with pull-up
    // and interrupt on falling edge

    gpio_config_t ioCfg =
    {
        .pin_bit_mask = 1ULL << INPUT_GPIO,
        .mode = GPIO_MODE_INPUT,
        .pull_up_en = GPIO_PULLUP_ENABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type = GPIO_INTR_NEGEDGE
    };
    gpio_config(&ioCfg);

    // Create a queue to handle GPIO events
    gpioEvtQueue = xQueueCreate(10, sizeof(uint32_t));

    // Start the task that will process GPIO events
    xTaskCreate(GPIONTask, "gpio_task", 2048, NULL, 10, NULL);

    // Install GPIO ISR service
    gpio_install_isr_service(0);

    // Hook ISR handler for specific GPIO
    gpio_isr_handler_add(INPUT_GPIO, GPIOISRHandler,
        (void*) INPUT_GPIO);
}

```

```

    printf("Waiting for GPIO %d falling edge interrupts...",
           INPUT_GPIO);
}

```

Question 6: Consult and analyze the ESP32-C6 GPIO & RTC and Queue APIs (respectively, <https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/api-reference/peripherals/gpio.html> and https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/api-reference/system/freertos_idf.html#queue-api). Study the provided code and complete the program to print the following information whenever a falling edge occurs in the push-button signal.

```

~$ Interrupt on GPIO 23
~$ Interrupt on GPIO 23
~$ Interrupt on GPIO 23
~$ Interrupt on GPIO 23
~$ ...

```

3.4 Controlling a PWM Output

The ESP-IDF LEDC library is the tool of choice for generating precise, efficient PWM signals on the ESP32 family of microcontrollers. Through it, it's possible to control the intensity of an LED by adjusting the signal's duty cycle, allowing, for example, the creation of smooth fade effects or simply regulating the brightness at different levels. In the case of an LED, LEDC enables a simple digital pin to act as a dimmer, opening the way for more dynamic and interactive interfaces in IoT projects and embedded systems.

```

#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/ledc.h"

#define LED_GPIO      GPIO_NUM_15
#define LEDC_CHANNEL  LEDC_CHANNEL_0
#define LEDC_TIMER     LEDC_TIMER_0
#define LEDC_MODE      LEDC_LOW_SPEED_MODE
#define LEDC_DUTY_RES  LEDC_TIMER_13_BIT // 13-bit resolution
#define LEDC_FREQUENCY 5000              // 5 kHz PWM

```



```

void app_main(void)
{
    // 1 - Configure timer
    ledc_timer_config_t ledcTimer =
    {
        .speed_mode = LEDC_MODE,
        .timer_num = LEDC_TIMER,
        .duty_resolution = LEDC_DUTY_RES,
        .freq_hz = LEDC_FREQUENCY,
        .clk_cfg = LEDC_AUTO_CLK
    };
    ledc_timer_config(&ledcTimer);

    // 2 - Configure channel
    ledc_channel_config_t ledcChannel =
    {
        .gpio_num = LED_GPIO,
        .speed_mode = LEDC_MODE,
        .channel = LEDC_CHANNEL,
        .intr_type = LEDC_INTR_DISABLE,
        .timer_sel = LEDC_TIMER,
        .duty = 0,
        .hpoint = 0
    };
    ledc_channel_config(&ledcChannel);

    uint32_t maxDuty = (1 << LEDC_DUTY_RES) - 1;
    uint32_t duty10 = maxDuty / 10;
    while (1)
    {
        ledc_set_duty(LEDC_MODE, LEDC_CHANNEL, duty10);
        ledc_update_duty(LEDC_MODE, LEDC_CHANNEL);
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

```

Question 7: Study the LEDC API and analyze how the example code works. The LEDC API description is available at: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/api-reference/peripherals/ledc.html>.

Question 8: Change the code for different duty-cycle values and observe the LED drive signal on the oscilloscope, confirming it. How do you justify the frequency used for the PWM?

Question 9: Change the code so that the LED exhibits a "breathing" effect: it gradually increases brightness to the maximum, then decreases to the minimum, repeating this process indefinitely.

3.5 ADC Configuration and Use in One-Shot Mode

The ADC's one-shot operating mode is used when precise, point-in-time readings of analog signals are required, without continuously maintaining active conversion. This mode allows the processor to request a measurement only when necessary, reducing power consumption and freeing up system resources, which is especially important in battery-powered IoT devices. In addition, one-shot simplifies integration with specific tasks, such as monitoring sensors at defined intervals or reading critical values in response to events, ensuring efficient, accurate processing of analog signals within embedded architectures.

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_adc/adc_oneshot.h"

#define ADC_CHANNEL ADC_CHANNEL_2 // GPIO2 (check pin for ESP32-C6)
#define ADC_UNIT ADC_UNIT_1 // Use ADC1
#define ADC_ATTEN ADC_ATTEN_DB_12 // 12 dB, ~3.3 V full-scale

void app_main(void)
{
    // ADC oneshot driver handle
    adc_oneshot_unit_handle_t adc1Handle;
    adc_oneshot_unit_init_cfg_t adc1InitCfg =
    {
        .unit_id = ADC_UNIT
    };
    adc_oneshot_new_unit(&adc1InitCfg, &adc1Handle);
```

```

// Configure channel
adc_oneshot_chan_cfg_t adcChanCfg=
{
    .atten = ADC_ATTEN,
    .bitwidth = ADC_BITWIDTH_DEFAULT // default = 12-bit
};
adc_oneshot_config_channel(adc1Handle, ADC_CHANNEL, &adcChanCfg);

while (1)
{
    int adcRawVal;
    float voltage;

    adc_oneshot_read(adc1Handle, ADC_CHANNEL, &adcRawVal);

    // Convert raw value to voltage (approximate, no calibration)
    voltage = (adcRawVal / 4095.0f) * (1.1f/0.25f);

    // Include here the code to print the digital value read
    // and the corresponding analogue value

    vTaskDelay(pdMS_TO_TICKS(1000));
}
}

```

Question 10: Study the ADC Oneshot API (https://docs.espressif.com/projects/espressif/en/stable/esp32c6/api-reference/peripherals/adc_oneshot.html), and complete the code so that the following information is printed every second:

```

~$ ADC Raw: 4095, Voltage: 3.422 V
~$ ADC Raw: 4095, Voltage: 3.422 V
~$ ADC Raw: 4095, Voltage: 3.422 V
~$ ADC Raw: 4095, Voltage: 3.422 V
~$ ...

```

Question 11: Observe the analog voltage value and compare it with the value obtained by your program. How do you justify the discrepancy?

Question 12: Implement a basic calibration mechanism and test it in practice, evaluating its operation.

Question 13: Study the ADC Calibration API (https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/api-reference/peripherals/adc_calibration.html), identify an alternative to the method used in the previous question, implement it, and compare its results with the previous basic calibration result.

Question 14: ESP-IDF allows you to create logs for different functionalities through the Logging Library. Repeat the previous exercises, replacing the use of the `printf` function with the `ESP_LOGI` macro.